Developing Apps on Qt

Part—3

In the previous article, we covered some important Qt non-GUI classes; I hope you experimented with others, since the secret to learning lies in experimenting. In this article, we will work on the backbone of GUIs—the signal and slot mechanism.

This signal and slot mechanism lets us communicate between objects. In everyday life, we see a lot of such examples; take, for example, a traffic signal—when it turns red, you stop your vehicle. The red light is the signal, and stopping the vehicle is the slot. When the traffic signal emits a red signal, you call the slot, i.e., you stop the vehicle.

To understand the mechanism properly, we need some GUI experience with Qt—our first GUI program, with signals/slot. Open Qt Creator. Navigate to *File → New File* or *Project → Qt Widget Project → Qt GUI Application* (Figure 1). Choose your location and name your new project (Figure 2). Choose the target as *Desktop* (Figure 3). Then you will be asked for class information. In the Base Class combo box, you will find three options, each used for a different purpose. We will cover these in the next article. For now, select the *QDialog* Base Class (Figure 4). Name your class, then click *Next*, and finally, *Finish*.

Once this is done, Qt Creator will open your newly created project (Figure 5), with the project window on the left side. The project will have one project file, *project-name.pro*; in this case, *1.pro* since I named my project "1". Below that are three folders: *Headers*, *Sources* and *Forms*, which contain the respective types of files. Double-click any file to open it. The class we named during project creation has three files: *dialog.h* (header file), *dialog.cpp* (CPP source) and *dialog.ui* (UI or form). Double-click the UI file to open the UI editor (Figure 6).

The left pane has different widgets that you can simply drag and drop on the dialogue box. In our first example, let us use one label and two push-buttons. The dialogue window should look like what's shown in Figure 7. For the sake of simplicity, let's not use any layout. Double-click the push-buttons and change their text to *'Count'* and *'Quit'*; double-click the label and clear the text. Clicking the *'Count'* button will increment a counter and display it on the label, and the *'Quit'* button will quit the application.

Now we will add a slot for the *'Quit'* button's *'clicked'* signal. Press F4 to edit signals/slots. Click the *'Quit'* button and drag your cursor to *Dialog*. When you release the mouse button, a new *'Configure Connection'* window will pop up (Figure 8). On the left are the signals that can be emitted by the widget; on the right are slots (actions, in every day language) that can be performed on a *QDialog* widget. On the left side, select *clicked()* and on the right, select *close()*. Thus, when *'Quit'* is clicked, the window will be closed. Press F3 to edit the widget section. The *'Count'* button also needs to be attached to a slot. Right-click it and choose *'Go to Slot'*. The pop-up window (Figure 9) 'Go to Slot' lists the signals emitted by the widget. Here, select *clicked()* and click *OK*. You will be taken to the *void Dialog::on_pushButton_clicked()* function in *dialog.cpp* (Figure 10). Edit this code as follows:

```
void Dialog::on_pushButton_clicked()
```